# 👤 User Management in Linux

Linux is a **multi-user operating system**, meaning multiple users can access and work on the same system simultaneously.
User management involves:

- Creating users
- Modifying users
- Deleting users
- Managing passwords
- Controlling login access
- Monitoring users

(🚫 We will NOT cover group management here.)

---

# 1️⃣ Types of Users in Linux

## 1. Root User

- Username: `root`
- UID: `0`
- Has complete control over the system
- Home directory: `/root`
- Can access and modify any file

Check root:

```
id root
```

---

## 2. Normal (Regular) Users

- Used for daily work
- Limited permissions
- Home directory: `/home/username`
- UID usually starts from 1000

Example:

```
/home/ashvi
```

---

## 3. System / Service Users

- Created automatically for services
- Usually no login shell
- UID range: 1–999 (varies by distro)

Example users:

- `daemon`
- `nobody`
- `apache`

---

# 2️⃣ Important User-Related Files

## 📄 /etc/passwd

Stores basic user account information.

View:

```
cat /etc/passwd
```

Example entry:

```
ashvi:x:1001:1001:Ashvi:/home/ashvi:/bin/bash
```

Field explanation:

```
username : password_placeholder : UID : GID : comment : home_dir : shell
```

Important:

- Password field shows `x`
- Actual encrypted password stored in `/etc/shadow`

---

## 📄 /etc/shadow

Stores hashed passwords.

Only root can read it:

```
sudo cat /etc/shadow
```

Contains:

- Hashed password
- Password expiry info
- Account expiry info

## Format:

```
username:hashed_password:last_change:min:max:warn:inactive:expire:reserved
```

## 🔍 Field Explanation

| Field | Meaning |
|---|---|
| username | Whose account |
| hashed_password | Secret scrambled password |
| last_change | Days since Jan 1, 1970 when password last changed |
| min | Minimum days before password can be changed |
| max | Maximum days password is valid |
| warn | Days before expiry to warn user |
| inactive | Grace period after expiry |
| expire | Account expiration date |

| | |
|---|---|
| reserved | Reserved |

### 🔐 Important Things to Remember

- Passwords are **hashed**, not encrypted.
- Only root can see this file.
- If the password field has `!` → account locked.
- If password field has `*` → login disabled.

### 🧠 Simple Comparison

| Symbol | Meaning | What Happens |
|---|---|---|
| `!` | Locked | Password was valid, but now blocked |
| `*` | Disabled | No password login possible at all |

---

## 📄 /etc/login.defs

The **UID (User ID)** and **GID (Group ID)** range information is stored in:

### 📊 What These Mean

| Variable | Purpose |
|---|---|
| `UID_MIN` | Minimum UID for normal users |

| | |
|---|---|
| `UID_MAX` | Maximum UID for normal users |
| `GID_MIN` | Minimum GID for normal groups |
| `GID_MAX` | Maximum GID for normal groups |
| `SYS_UID_MIN/MAX` | Range for system users |
| `SYS_GID_MIN/MAX` | Range for system groups |

---

### 🧠 Typical Linux UID Layout

| UID Range | Type |
|---|---|
| 0 | root |
| 1–999 | System users (services like apache, nginx, mysql) |
| 1000+ | Regular users |

---

# 3️⃣ Creating Users

There are two main commands:

---

## 🔹 useradd (Basic Method)

```
sudo useradd username
```

⚠️ By default:

- May NOT create home directory

- No password set
- Cannot login until password is set

---

### 🔹 useradd with Options

```
sudo useradd -m -s /bin/bash ashvi
```

Options:

- `-m` → Create home directory
- `-s` → Specify shell
- `-c` → Add comment

Example:

```
sudo useradd -m -s /bin/bash -c "DevOps Engineer" ashvi
```

---

### 🔹 adduser (Recommended – Interactive)

```
sudo adduser ashvi
```

✔ Creates home directory
✔ Prompts for password
✔ Asks for user details

More user-friendly (Debian/Ubuntu systems).

---

## 4️⃣ Setting and Changing Password

### Set password for user:

```
sudo passwd ashvi
```

### Change your own password:

```
passwd
```

### Force password change on next login:

```
sudo passwd -e ashvi
```

---

# 5️⃣ Viewing User Information

### Check current user:

```
whoami
```

### Check user ID:

```
id ashvi
```

Output example:

```
uid=1001(ashvi)
```

### List logged-in users:

```
who
```

```
w
```

---

# 6️⃣ Switching Users

## Switch to another user:

```
su ashvi
```

## Switch to root:

```
su -
```

## Switch using sudo:

```
sudo -i
```

---

## 1️⃣ What is sudo?

sudo = **Super User DO**

It allows permitted users to execute commands as:

- root (most common)
- Another user
- A specific group

Instead of logging in as root, users run:

```
sudo command
```

**Access is controlled by /etc/sudoers.**

---

## 2️⃣ Important Rule: Always Edit with visudo

**Never edit /etc/sudoers directly with vi or nano.**

Use:

```
sudo visudo
```

**Why?**

- It checks syntax before saving -> Prevents configuration errors
- Prevents locking yourself out -> Saves you from losing admin access
- Uses file locking -> Stops multiple people breaking the file

---

## 3️⃣ User Privilege Specification (Most Important Part)

General format of a sudoers rule:

**root ALL=(ALL:ALL) ALL**

**Meaning:**

```
user   host=(run_as_user:run_as_group)   command
```

---

## 4️⃣ Group-Based Access

Instead of adding users one by one, you use groups.

Example:

**%sudo ALL=(ALL:ALL) ALL**

The `%` means group.

This allows all users in the `sudo` group full sudo access.

**On Ubuntu/Debian:**

- Group is `sudo`

**On RHEL/CentOS:**

- Group is `wheel`

**Example:**

%wheel  ALL=(ALL) ALL

---

## 5️⃣ NOPASSWD Option

Allows running without password.

**Example:**

ashvi ALL=(ALL) NOPASSWD: /bin/systemctl restart nginx

**Now ashvi can restart nginx without password:**

sudo systemctl restart nginx

⚠️ Use carefully — security risk.

---

## 6️⃣ Command Restrictions

You can limit specific commands.

Example:

**ashvi ALL=(ALL) /bin/systemctl restart nginx, /bin/systemctl status nginx**

**User can:**

- Restart nginx
- Check status
- Nothing else

---

# 7️⃣ Modifying Users (usermod)

## Change username:

```
sudo usermod -l newname oldname
```

---

## Change home directory:

```
sudo usermod -d /home/newdir -m ashvi
```

`-m` moves existing files.

---

## Change login shell:

```
sudo usermod -s /bin/zsh ashvi
```

Common shells:

- `/bin/bash`
- `/bin/sh`
- `/bin/zsh`
- `/sbin/nologin` (disable login)

---

# 8️⃣ Locking and Unlocking Users

## Lock user account:

```
sudo usermod -L ashvi
```

OR

```
sudo passwd -l ashvi
```

### Unlock:

```
sudo usermod -U ashvi
```

---

# 9️⃣ Deleting Users

### Delete user only:

```
sudo userdel ashvi
```

### Delete user + home directory:

```
sudo userdel -r ashvi
```

⚠️ Always backup important data before deleting.

---

# 🔟 Disabling User Login

### Disable shell:

```
sudo usermod -s /sbin/nologin ashvi
```

OR

```
sudo usermod -s /bin/false ashvi
```

Users cannot log in.

---

# ➕Including chage (Password Aging) & Default User Creation Settings

This guide focuses only on **user management** (no group management), with special emphasis on:

- ✅ `chage` command (password aging control)
- ✅ Default user creation settings
- ✅ Account expiry & security policies

---

# 1️⃣Understanding User Account Lifecycle

When a user is created in Linux:

1. Entry is added to `/etc/passwd`
2. Password is stored in `/etc/shadow`
3. Home directory is created (if `-m` used)
4. Default settings are applied from:
    - `/etc/login.defs`
    - `/etc/default/useradd`
    - `/etc/skel`

---

# 2️⃣ Password Aging & Expiry – chage Command (Very Important 🔥)

## 👉 What is chage?

`chage` = **Change Age**

It is used to manage:

- Password expiry
- Account expiry
- Password warning days
- Minimum & maximum password validity

---

## 🔹 Check Current Password Aging Information

```
chage -l username
```

Example:

```
chage -l ashvi
```

Example output:

```
Last password change                                    : Feb 16, 2026
Password expires                                        : Mar 18, 2026
Password inactive                                       : never
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 30
Number of days of warning before password expires       : 7
```

---

# 3️⃣ Understanding chage Parameters

---

## 🔹 1. Set Maximum Password Validity

```
sudo chage -M 30 username
```

Meaning:

- Password will expire after 30 days.

---

## 🔹 2. Set Minimum Days Between Password Changes

```
sudo chage -m 7 username
```

User cannot change password before 7 days.

---

## 🔹 3. Set Warning Days Before Expiry

```
sudo chage -W 5 username
```

User will get warning 5 days before password expires.

---

## 🔹 4. Set Account Expiry Date

```
sudo chage -E 2026-12-31 username
```

After that date:

- User cannot login.

---

## 🔹 5. Force Password Change at Next Login

```
sudo chage -d 0 username
```

OR

```
sudo passwd -e username
```

User must change password immediately.

---

## 🔹 6. Disable Account Expiry

```
sudo chage -E -1 username
```

Removes account expiration.

---

## 🔹 7. Change All at Once (Interactive Mode)

```
sudo chage username
```

**It will ask:**

Minimum Password Age

Maximum Password Age

Password Expiration Warning

Account Expiration Date

---

# 4️⃣ Complete `chage` Example (Real Scenario)

Set company policy:

- Password expires in 60 days
- Minimum 7 days before change
- Warning 10 days before expiry
- Account expires on 31 Dec 2026

```
sudo chage -M 60 -m 7 -W 10 -E 2026-12-31 ashvi
```

Verify:

```
chage -l ashvi
```

---

# 5️⃣ What Happens When Passwords Expire?

- User gets warning
- After expiry → login denied
- Admin must reset password

---

# 6️⃣ Difference: Account Expiry vs Password Expiry

| Feature | Password Expiry | Account Expiry |
| --- | --- | --- |

| | | |
|---|---|---|
| Controlled by | -M option | -E option |
| Effect | Must change password | Cannot login at all |
| Common use | Security policy | Temporary employees |

---

# 7️⃣ Default User Creation Settings (Very Important 🔥)

When you create a user, Linux uses default settings from these files:

---

## 📄 1. `/etc/login.defs`

Controls:

- Default UID range
- Password aging defaults
- Home directory creation settings

Check:

```
cat /etc/login.defs
```

Important parameters:

### 🔹 UID Range

```
UID_MIN 1000
UID_MAX 60000
```

---

### 🔹 Password Aging Defaults

```
PASS_MAX_DAYS   99999
PASS_MIN_DAYS   0
PASS_WARN_AGE   7
```

Meaning:

- Password valid for 99999 days
- No minimum days
- 7-day warning

---

## 🎯1️⃣ Set Default Password Aging Policy (For All NEW Users)

If you want to enforce the **same password policy for all users**, then this is controlled by: **/etc/login.defs**

1. **Edit it: sudo vi /etc/login.defs**
2. Look for these parameters:

PASS_MAX_DAYS 90

PASS_MIN_DAYS 7

PASS_WARN_AGE 7

3. **Meaning:**

| **Setting** | **Effect** |
|---|---|
| PASS_MAX_DAYS 90 | Password expires in 90 days |
| PASS_MIN_DAYS 7 | User must wait 7 days before changing |
| PASS_WARN_AGE 7 | Warn 7 days before expiry |

⚠ **Important:**
This applies only to **new users created after the change**.

---

## 🔄2️⃣ Apply Policy to Existing Users

1. To enforce for all existing users:

Example: sudo chage -M 90 -m 7 -W 7 username

2. **To apply for all normal users (UID ≥ 1000):**

```
for user in $(awk -F: '$3 >= 1000 {print $1}' /etc/passwd); do
  sudo chage -M 90 -m 7 -W 7 $user
done
```

**Now all users follow the same aging policy.**

---

## 🔐3️⃣ Enforce Password Complexity (Strong Password Rules)

Password length, special characters, etc. are controlled by PAM.

**Modern Linux uses: /etc/security/pwquality.conf**

1. **Edit: sudo vi /etc/security/pwquality.conf**
2. **Example strong policy:**

```
minlen = 12
dcredit = -1
ucredit = -1
lcredit = -1
ocredit = -1
maxrepeat = 3
```

3. **Meaning:**

| Setting | Meaning |
| --- | --- |
| minlen=12 | Minimum 12 characters |
| dcredit=-1 | At least 1 digit |
| ucredit=-1 | At least 1 uppercase |
| lcredit=-1 | At least 1 lowercase |
| ocredit=-1 | At least 1 special character |
| maxrepeat=3 | No more than 3 repeated characters |

---

## 🔁4️⃣ Prevent Password Reuse

1. Edit PAM config:

**On RHEL/CentOS:/etc/pam.d/system-auth**

**On Ubuntu: /etc/pam.d/common-password**

2. **Add or ensure:**

password required pam_pwhistory.so remember=5

3. **Meaning:**

User cannot reuse last 5 passwords.

---

### 🚫5️⃣ Lock Account After Failed Attempts

1. Edit: **/etc/pam.d/system-auth**
2. Add:

**auth required pam_faillock.so preauth silent deny=5 unlock_time=900**

**auth required pam_faillock.so authfail deny=5 unlock_time=900**

3. Meaning:
    - Lock after 5 failed attempts
    - Unlock after 15 minutes (900 seconds)

---

# 📄 2. `/etc/default/useradd`

Contains default settings for `useradd`.

Check:

```
cat /etc/default/useradd
```

Example:

```
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/bash
SKEL=/etc/skel
CREATE_MAIL_SPOOL=yes
```

**📋 Field-by-Field Explanation**

| Parameter | Meaning |
| --- | --- |
| **GROUP=100** | Default primary group ID assigned to new users |

| | |
|---|---|
| **HOME=/home** | Base directory where user home directories are created |
| **INACTIVE=-1** | Days after password expiry before account is disabled |
| **EXPIRE=** | Account expiration date (empty = never expires) |
| **SHELL=/bin/bash** | Default login shell |
| **SKEL=/etc/skel** | Skeleton directory copied into new user's home |
| **CREATE_MAIL_SPOOL=yes** | Whether to create mail file in `/var/mail/username` |

---

## 📄 3. `/etc/skel`

This directory contains default files copied to new users.

Check:

```
ls -a /etc/skel
```

Common files:

```
.bashrc
.bash_profile
.profile
```

When new user is created:

```
sudo useradd -m ashvi
```

Contents of `/etc/skel` are copied into:

```
/home/ashvi
```

---

# 8️⃣ Changing Default Settings for New Users

---

### 🔹 Change Default Shell

```
sudo useradd -D -s /bin/zsh
```

Check:

```
useradd -D
```

---

### 🔹 Change Default Expiry

```
sudo useradd -D -e 2026-12-31
```

---

### 🔹 Change Default Inactive Days

```
sudo useradd -D -f 30
```

Meaning:
Account disabled 30 days after password expiry.

---

# 9️⃣ Checking Default Settings

```
useradd -D
```

Example output:

```
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
```

```
SHELL=/bin/bash
```

---

# 🔟 Security Best Practices

✔ Set password expiry (60–90 days)
✔ Set minimum password age
✔ Set warning days
✔ Disable inactive accounts
✔ Use account expiry for temporary users
✔ Regularly check `chage -l`

---

# 📌 Quick Command Summary

| Command | Purpose |
|---|---|
| chage -l | Check aging |
| chage -M | Set max days |
| chage -m | Set min days |
| chage -W | Warning days |
| chage -E | Account expiry |
| chage -d 0 | Force password change |
| useradd -D | Show defaults |
| useradd -D -s | Change default shell |

---